Cahier de sp��cification technique ��� Rescue Grid

Cahier de spécification technique — Rescue Grid

1. Acteurs

Acteur	Type	Rôle
Abonné	Humain	Ménage/PME, utilisateur final côté terrain
Technicien	Humain	Opérateur (SOCADEL), intervient sur le réseau
Administrateur	Humain	Supervise la santé des boîtiers déployés
Fournisseur de connectivité (Wi-Fi/GSM)	Externe passif	Déclenche les comportements de bascule en cas de panne

2. Périmètre du “système” — 3 sous-systèmes

Le “système” Rescue Grid se décompose en trois sous-systèmes distincts, chacun avec ses propres cas d’utilisation internes :

Sous-système	Rôle
Boîtier ESP32	Acquisition, traitement local (edge), transmission
Backend	Authentification, agrégation, corrélation, stockage, API
Frontend	Interface(s) de visualisation — vue Admin/Technicien et vue Abonné

Décision tranchée : - Interface Admin/Technicien → **application web** uniquement - Interface Abonné → **application web ET mobile**

3. Liste des cas d’utilisation par acteur

Abonné - UC1 : Consulter l’état de sa ligne en temps réel - UC2 : Recevoir une notification push d’anomalie - UC3 : Recevoir un SMS d’alerte en cas de coupure Wi-Fi/GPRS - UC4 : Consulter l’historique des incidents de son domicile - UC5 : Configurer ses seuils de notification

Technicien - UC6 : Localiser une panne sur la carte de supervision - UC7 : Consulter le diagnostic de corrélation (locale/quartier/secteur) - UC8 : Clôturer un incident après intervention - UC9 : Exporter un rapport d’incident - UC23 : Capturer la position GPS d’un boîtier à l’installation

Administrateur - UC10 : Consulter l’état de santé des boîtiers déployés - UC11 : Recevoir une alerte de boîtier hors-ligne - UC12 : Ajouter/retirer un boîtier du parc supervisé - UC13 : Consulter les logs d’authentification des boîtiers

Boîtier ESP32 (acteur système) - UC14 : Échantillonner la tension et la fréquence - UC15 : Détecter un dépassement de seuil - UC16 : Transmettre une trame de mesure au serveur - UC17 : Basculer la transmission vers SMS/GSM - UC18 : Émettre un rapport d’incident final (mode batterie)

Serveur de corrélation / Backend (acteur système) - UC19 : Authentifier une trame entrante *(vérifier que le paquet de données vient bien d’un boîtier légitime enregistré, pas d’un faux appareil)* - UC20 : Agréger les mesures de plusieurs boîtiers *(rassembler les données de plusieurs boîtiers d’une même zone pour avoir une vue d’ensemble, pas boîtier par boîtier)* - UC21 : Classifier l’origine d’une anomalie *(décider si la panne est locale/un abonné, sectorielle/un quartier-transformateur, ou globale/centrale-transport, à partir des données agrégées)* - UC22 : Mettre à jour la cartographie dynamique *(afficher sur la carte de supervision l’état et la localisation des pannes détectées)*

4. Diagramme de cas d’utilisation

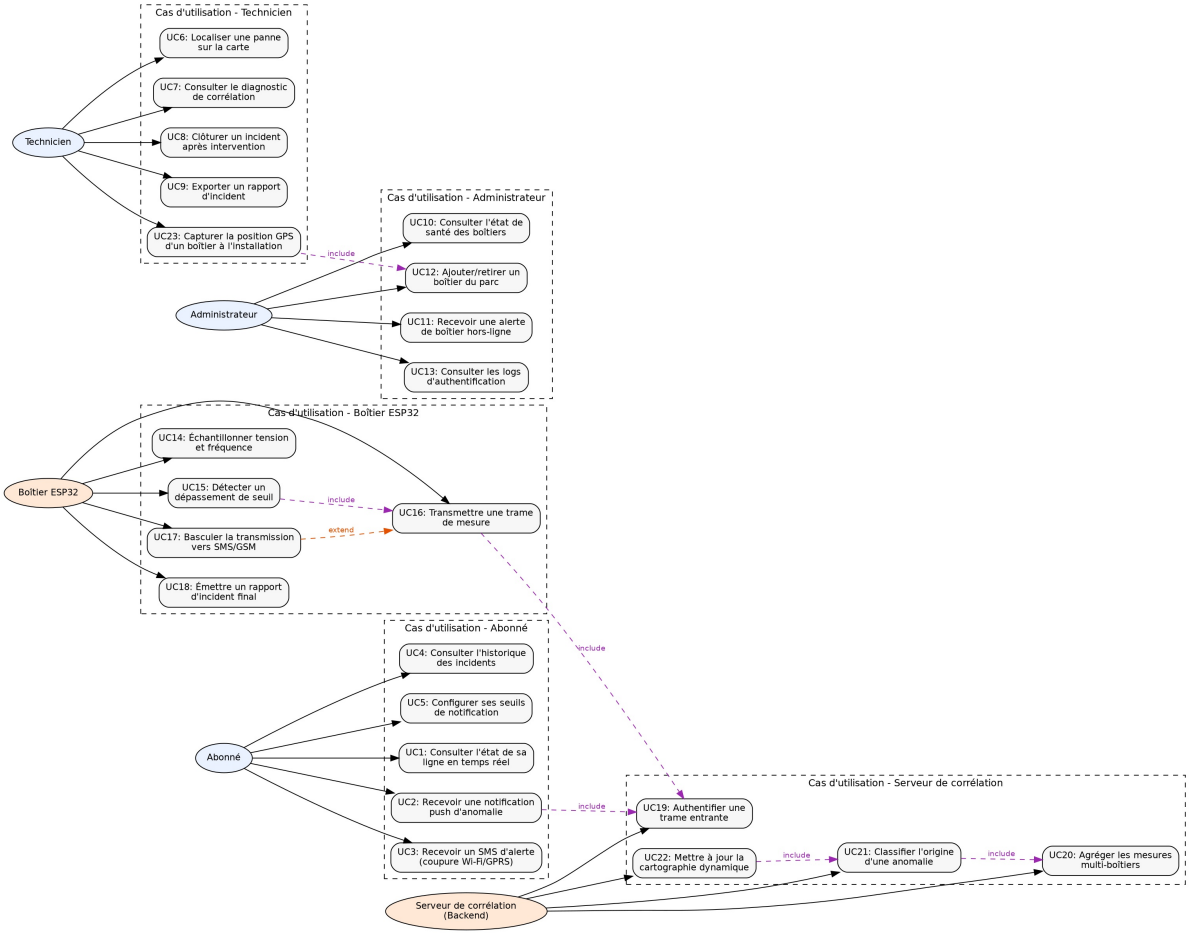


Diagramme 1

5. Répartition des cas d’utilisation par sous-système

Cas d’utilisation	Sous-système responsable
UC1-UC5 (Abonné)	Frontend (vue Abonné) — consomme les données du Backend
UC6-UC9 (Technicien)	Frontend (vue Admin/Technicien) — consomme les données du Backend
UC10-UC13 (Administrateur)	Frontend (vue Admin/Technicien) — consomme les données du Backend
UC14-UC18 (Boîtier ESP32)	Boîtier ESP32
UC19-UC22 (Serveur de corrélation)	Backend

> À noter : les acteurs humains (Abonné, Technicien, Admin) interagissent **exclusivement via le Frontend** — ils n’ont jamais d’interaction directe avec le Backend ou le Boîtier. Ça doit rester vrai dans les diagrammes de séquence : toute requête humaine passe par une couche Frontend qui appelle le Backend.

6. Points de dépendance identifiés entre sous-systèmes

- UC16 (Transmettre une trame) dépend de UC19 (Authentifier) côté serveur : chaque trame doit être signée/authentifiée avant traitement.
- UC17 (Bascule SMS) est un cas d’extension de UC16, déclenché uniquement en cas d’échec de la transmission Wi-Fi/GPRS.
- UC21 (Classifier l’origine) dépend de UC20 (Agréger) : la classification ne peut se faire qu’après agrégation d’au moins deux sources ou d’un historique suffisant.
- UC2 (Notification push abonné) dépend indirectement de UC19 côté serveur — pas de notification sans trame authentifiée.

7. Besoins fonctionnels détaillés

Un besoin fonctionnel = une règle précise et testable, dérivée d’un cas d’utilisation. Verbes concrets uniquement (mesurer, comparer, transmettre, afficher…), jamais “gérer” ou équivalent.

UC14 — Échantillonner la tension et la fréquence - BF-01 : Le boîtier mesure la tension et la fréquence en continu, à haute fréquence d’échantillonnage. - BF-02 : Le boîtier convertit chaque valeur brute de l’ADC en valeur réelle à l’aide de la table de calibration (`CalibPoint`).

- *(taille exacte de la fenêtre d'échantillonnage avant calcul RMS : à préciser plus tard)*

UC15 — Détecter un dépassement de seuil - BF-03 : Le système calcule la valeur RMS et la fréquence sur la fenêtre d'échantillonnage. - BF-04 : Le système compare la valeur RMS et la fréquence à des seuils de sécurité prédéfinis. - BF-05 : Si un seuil est dépassé, le système ouvre une fenêtre de confirmation de 1 à 3 secondes. - BF-06 : Le système confirme l'anomalie si le dépassement persiste pendant toute la fenêtre de confirmation ; sinon il journalise l'événement comme faux positif sans le transmettre. - BF-07 : Le système classe chaque anomalie confirmée dans une des trois catégories : surtension, sous-tension, ou dérive de fréquence.

UC16 — Transmettre une trame de mesure - BF-08 : Le boîtier envoie une trame contenant la mesure, l'horodatage et le type d'anomalie au backend via MQTT. - *(structure exacte de la trame et logique détaillée : à préciser au moment du diagramme dédié à UC16)*

UC17 — Basculer la transmission vers SMS/GSM - BF-09 : Le boîtier bascule automatiquement vers SMS/GSM si la transmission Wi-Fi/GPRS échoue. - *(délai exact de détection de l'échec et de bascule : à préciser)*

UC18 — Émettre un rapport d'incident final (mode batterie) - BF-10 : Le boîtier envoie un rapport final incluant l'heure exacte et la nature de la coupure avant l'extinction complète de la batterie.

UC19 — Authentifier une trame entrante - BF-11 : Le backend vérifie la signature/le token de chaque trame reçue avant de la traiter. - BF-12 : Le backend rejette et journalise toute trame non authentifiée sans la transmettre au moteur de corrélation.

UC20 — Agréger les mesures de plusieurs boîtiers - BF-13 : Le backend regroupe les trames reçues par proximité géographique (rayon de X mètres autour de chaque boîtier — seuil à calibrer empiriquement) et par fenêtre temporelle commune.

UC21 — Classifier l'origine d'une anomalie - BF-14 : Le backend classe chaque anomalie agrégée comme locale, sectorielle, ou globale, avec un taux de classification correcte de 98-99%. - BF-14bis : La classification repose sur une corrélation géo-temporelle, sans nécessiter de donnée de topologie réseau (transformateur/ligne) dans cette version : - **Locale** : un seul boîtier signale une anomalie ; aucun boîtier dans le rayon de proximité ne signale rien d'anormal au même moment. - **Sectorielle** : plusieurs boîtiers regroupés dans un rayon géographique limité signalent une anomalie simultanément, pendant que des boîtiers hors de ce rayon restent normaux. - **Globale** : des boîtiers dispersés sur plusieurs zones géographiques distinctes et non adjacentes signalent tous une anomalie en même temps. - *(Piste future, hors périmètre de cette version) : affiner la classification sectorielle pour distinguer transformateur vs tronçon de ligne, une fois qu'une donnée de topologie réseau (rattachement boîtier → transformateur) sera disponible — cf. classe* `Transformateur` / `ZoneDistribution` *à ajouter ultérieurement au diagramme de classes.*

UC22 — Mettre à jour la cartographie dynamique - BF-15 : Le backend met à jour la carte de supervision en moins de 200ms après chaque nouvelle classification.

UC1 — Consulter l'état de sa ligne en temps réel - BF-16 : L'abonné visualise la tension et la fréquence actuelles de sa ligne sur l'interface Frontend (web/mobile).

UC2 — Recevoir une notification push d'anomalie - BF-17 : Le système envoie une notification push à l'abonné dans les 30 secondes suivant la confirmation d'une anomalie.

UC3 — Recevoir un SMS d'alerte - BF-18 : Le système envoie un SMS à l'abonné si la notification push échoue ou si le canal Wi-Fi/GPRS est indisponible.

UC4 — Consulter l'historique des incidents - BF-19 : L'abonné consulte la liste des incidents passés de son domicile, avec date, type, et durée.

UC5 — Configurer ses seuils de notification - BF-20 : L'abonné modifie les seuils de tension/fréquence déclenchant une notification, dans une plage de valeurs autorisée par le système.

UC6 — Localiser une panne sur la carte - BF-21 : Le technicien visualise la localisation géographique de chaque panne active sur la carte de supervision.

UC7 — Consulter le diagnostic de corrélation - BF-22 : Le technicien consulte la classification (locale/sectorielle/globale) associée à chaque panne affichée.

UC8 — Clôturer un incident après intervention - BF-23 : Le technicien marque un incident comme résolu, avec horodatage de clôture.

UC9 — Exporter un rapport d'incident - BF-24 : Le technicien exporte un rapport d'incident (format à préciser : PDF/CSV).

UC23 — Capturer la position GPS d'un boîtier à l'installation - BF-29 : Le technicien scanne le QR code (ou saisit l'ID) du boîtier à installer, ouvrant un formulaire web mobile-friendly. - BF-30 : Le formulaire capture la position via l'API Geolocation du navigateur (`navigator.geolocation.getCurrentPosition()`), avec possibilité d'ajouter une description texte libre (`adresseDescription`). - BF-31 : Si un autre boîtier du même abonné a déjà été positionné lors de la même session d'installation, le technicien peut réutiliser cette position plutôt que de recapturer (bouton dédié) — évite le bruit GPS répété pour un même site. - BF-32 : Pour des abonnés distincts au même immeuble (ex: plusieurs locataires installant chacun leur boîtier indépendamment), chaque boîtier capture sa propre position sans mécanisme de réutilisation — les coordonnées, bien que légèrement différentes (bruit GPS), restent suffisamment proches pour être correctement regroupées par la logique de corrélation géo-temporelle (BF-13). - BF-33 : La validation du formulaire envoie les données (idBoitier, latitude, longitude, adresseDescription) au backend via une requête authentifiée (identité du technicien, distincte de l'authentification des boîtiers UC19).

UC10 — Consulter l'état de santé des boîtiers - BF-25 : L'administrateur visualise le niveau de batterie, la connectivité et l'horodatage de dernière transmission de chaque boîtier déployé.

UC11 — Recevoir une alerte de boîtier hors-ligne - BF-26 : Le système notifie l'administrateur si un boîtier n'a transmis aucune donnée depuis un délai configurable.

UC12 — Ajouter/retirer un boîtier du parc supervisé - BF-27 : L'administrateur enregistre ou désactive un boîtier dans le système, avec génération/révocation de son identifiant d'authentification.

UC13 — Consulter les logs d'authentification - BF-28 : L'administrateur consulte l'historique des tentatives d'authentification (réussies et rejetées) de chaque boîtier.

8. Besoins non fonctionnels détaillés

Résilience - *(Fonctionnalité différée — implémentation reportée à une phase ultérieure)* Le boîtier doit redémarrer automatiquement (watchdog) en moins de X secondes après un bug logiciel ou une perturbation électromagnétique, sans perte de la configuration de calibration. - Le backend doit rester disponible même en cas de perte d'un nœud serveur (réplication/redondance à préciser selon l'infrastructure cloud choisie).

Précision de mesure - La tension mesurée doit avoir une précision de **98-99%** (marge d'erreur ≤ 1-2%) par rapport à un multimètre de référence étalonné. - Le taux de classification correcte de l'origine d'une panne (locale/sectorielle/globale) doit être supérieur à **98-99%**.

Latence - Le système doit notifier une anomalie majeure en **30 secondes maximum** entre la détection par le boîtier et la réception effective par l'opérateur/technicien et par l'abonné.

Autonomie énergétique - Le boîtier doit garantir un fonctionnement sur batterie de secours d'au moins 3 heures sans alimentation secteur, avec transmission garantie du rapport d'incident final (mode "Dernier Souffle") avant extinction complète.

Basculement réseau (communication) - La commutation entre Wi-Fi/GPRS et SMS/GSM doit être transparente pour l'utilisateur, sans perte de trame, en moins de X secondes après détection de l'échec du canal principal.

Sécurité des données - Chaque trame transmise doit être authentifiée pour empêcher qu'un faux boîtier n'injecte de fausses alertes dans le système. - Les échanges Frontend-Backend doivent être chiffrés (HTTPS/TLS minimum).

Robustesse électrique - Le circuit d'alimentation du boîtier doit rester opérationnel face à des surtensions transitoires allant jusqu'à 400V, sans destruction du capteur.

Performance de traitement (Edge Computing) - L'ESP32 doit traiter plusieurs milliers de points d'échantillonnage par seconde sans latence perceptible sur le calcul RMS/fréquence.

Scalabilité et disponibilité serveur - Le backend doit gérer la réception simultanée de données provenant de centaines à milliers de boîtiers, avec un temps de réponse inférieur à 200ms, sans dégradation de la mise à jour de la cartographie.

Ergonomie / Utilisabilité - Les interfaces Frontend (Admin et Abonné) doivent permettre à un utilisateur non technique de comprendre l'état de sa ligne ou d'une panne en moins de X secondes de consultation, sans formation préalable. - Le design doit rester visuellement clair et attractif (hiérarchie visuelle simple, code couleur cohérent pour les niveaux d'alerte).

Coût - La consommation de données mobiles doit être optimisée (protocole MQTT léger) pour rester abordable pour l'utilisateur final au Cameroun.

9. Diagrammes de séquence

9.1 Boîte noire — chaîne d'alerte complète (vue globale)

Diagramme unique montrant les échanges entre sous-systèmes traités comme des blocs opaques (pas de logique interne visible). Couvre la chaîne UC15 → UC16 → UC19 → UC21 → UC2, avec le cas alternatif de bascule SMS (UC17).

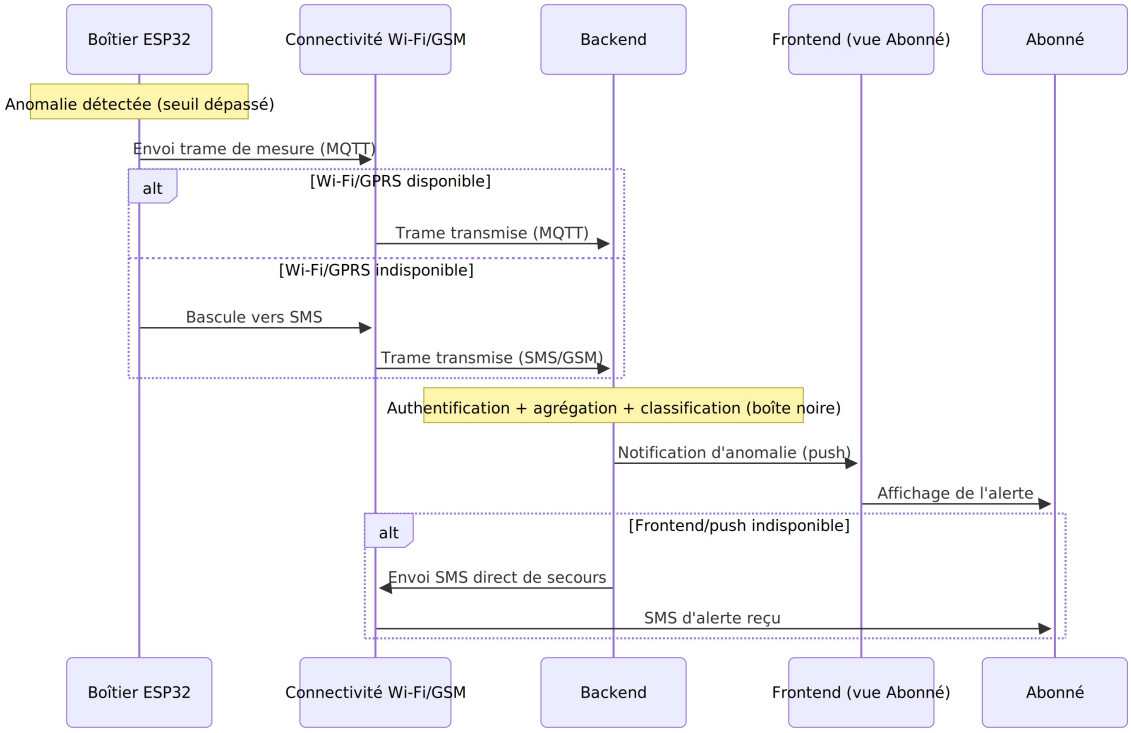


Diagramme 2

9.2 Boîte blanche — un diagramme par cas d’utilisation

Ordre retenu : 1) opérations du Boîtier, 2) opérations du Backend, 3) opérations du Frontend. Les cas qui jumellent deux sous-systèmes sont annotés.

1. Boîtier - UC14 + UC15 (Échantillonnage + détection de seuil) — *interne boîtier uniquement* - UC16 (Transmission de trame) — *jumelle Boîtier + Backend* (authentification côté backend) - UC17 (Bascule SMS) — *jumelle Boîtier + Connectivité*, extension de UC16 - UC18 (Rapport d’incident final, mode batterie) — *jumelle Boîtier + Backend*

2. Backend - UC19 (Authentifier une trame) — *jumelle Boîtier + Backend* (déjà couvert avec UC16, à détailler côté backend) - UC20 + UC21 (Agrégation + classification) — *interne backend uniquement* - UC22 (Mise à jour cartographie) — *jumelle Backend + Frontend* (le frontend consomme la carte)

3. Frontend - UC2 (Notification push abonné) — *jumelle Backend + Frontend* - UC1, UC4 (Consulter état ligne / historique, abonné) — *jumelle Frontend + Backend* (lecture API) - UC6, UC7 (Localiser panne, consulter diagnostic, technicien) — *jumelle Frontend + Backend* - UC10, UC11 (État de santé boîtiers, alerte hors-ligne, admin) — *jumelle Frontend + Backend* - UC5, UC8, UC9, UC12, UC13 (configuration, clôture, export, gestion parc, logs) — *jumelle Frontend + Backend*, à traiter en fin de liste

On commence par UC14 + UC15.

9.2.1 UC14 + UC15 — Échantillonnage et détection de seuil (interne Boîtier)

Composants internes du Boîtier ouverts pour ce diagramme : - **Boucle principale** (scheduler) - **ADC** (capteur brut) - **Module de calibration** (table `CalibPoint`, conversion brut → valeur réelle) - **Module de traitement du signal** (calcul RMS, fréquence) - **Module de filtrage** (confirmation par persistance sur fenêtre 1-3s, anti-faux-positifs) - **Module de décision** (comparaison aux seuils de sécurité)

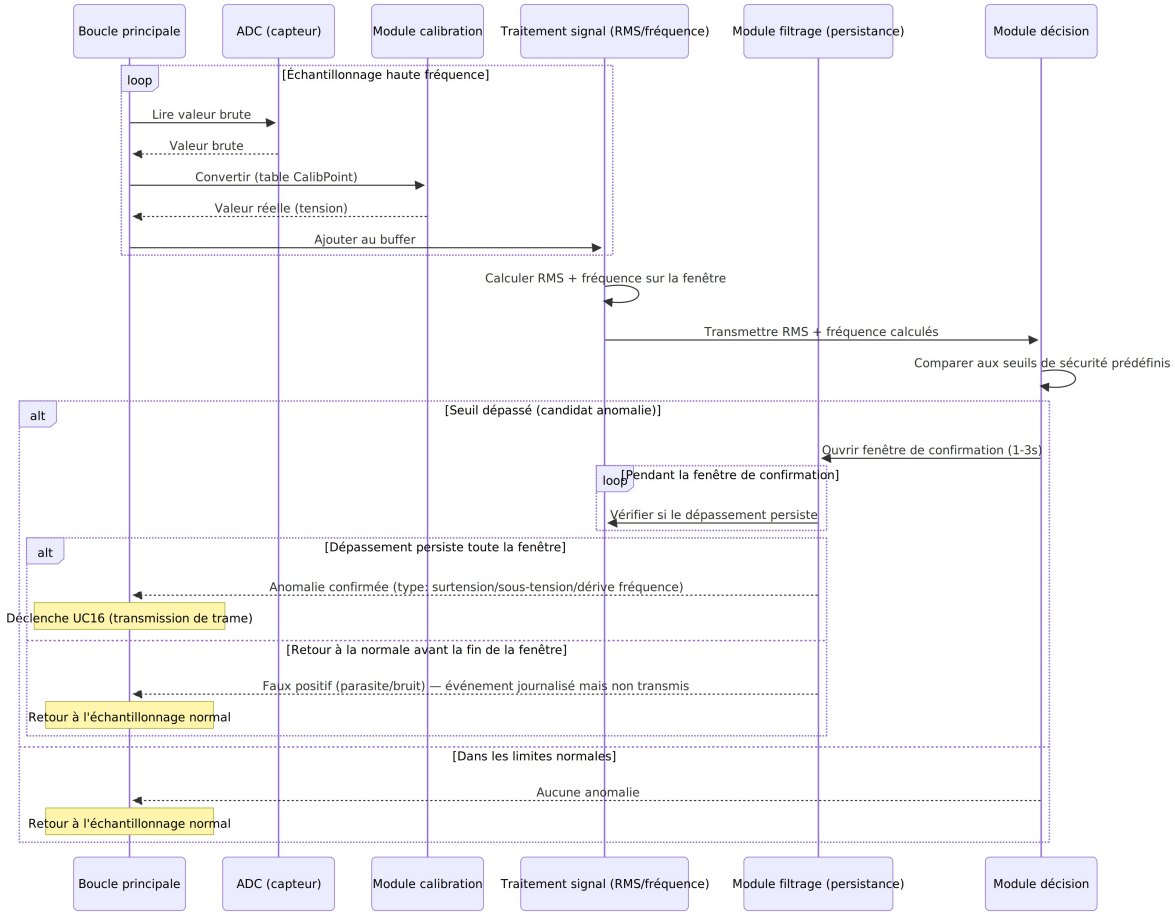


Diagramme 3

Note de conception : le filtre valide par **persistance du dépassement**, pas par tendance progressive — une coupure ou surtension brutale (saut immédiat) reste détectée correctement, puisqu'elle persiste elle aussi pendant la fenêtre de confirmation. Seuls les pics transitoires qui reviennent à la normale avant la fin de la fenêtre sont filtrés comme faux positifs.

Piste future (hors périmètre de ce diagramme) : un module IA embarqué (TinyML) pourrait compléter ce filtre pour détecter les micro-oscillations annonciatrices d'une panne (maintenance prédictive, cf. cahier des charges 2.2.1) — mécanisme distinct de la détection immédiate ci-dessus.

10. Diagramme d'état-transition

10.1 Boîtier ESP32 (uniquement)

États dérivés du cahier des charges (résilience, autonomie batterie, watchdog, bascule réseau) :

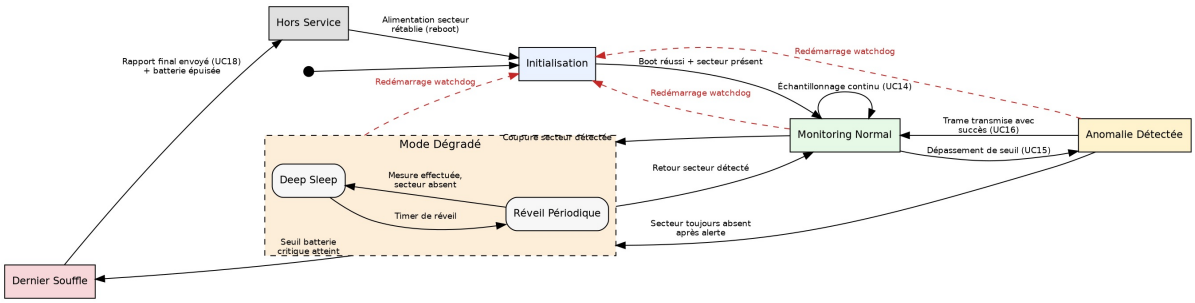


Diagramme 4

> [Différé] **Fonctionnalité différée** : le mécanisme de redémarrage watchdog (transitions en pointillé ci-dessus) est documenté ici pour la cohérence du cycle de vie, mais son implémentation est reportée à une phase ultérieure du projet, par choix de priorisation (voir section 8 — Résilience).

Points validés / en attente : - [Validé] Redémarrage watchdog accessible depuis n'importe quel état actif — validé - [Validé] `DernierSouffle` comme état transitoire obligatoire avant `HorsService` (KPI 100%) — validé - [En attente] Transition explicite `ModeDegrade → DernierSouffle` pendant un `ReveilPeriodique` (vs. implicite au niveau de l'état composite) — **en attente, à trancher plus tard**

10.2 Incident (proposition — à valider)

Cycle de vie proposé : `Détecté` → `Notifié` → `En cours d'intervention` → `Clôturé`, avec une branche alternative vers `Faux Positif` si l'anomalie est invalidée après vérification humaine.

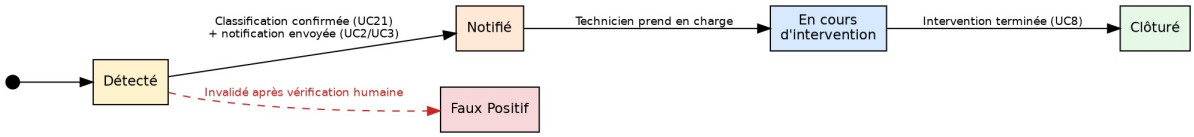


Diagramme état-transition Incident

11. Diagramme de classes

11.1 Attributs identifiés par classe (brouillon — à vérifier)

Classe **`Trame`** *(dérivée de UC16)* - `idTrame` : identifiant unique de la trame - `idBoitier` : référence au boîtier émetteur - `horodatage` : date/heure précise de la mesure - `tensionRMS` : valeur de tension mesurée - `frequence` : valeur de fréquence mesurée - `typeAnomalie` : énumération {surtension, sous-tension, dérive_fréquence, normal} - `canalTransmission` : énumération {wifi_mqtt, sms_gsm} - `signatureAuthentification` : jeton/signature utilisé pour UC19 - `statutTransmission` : énumération {envoyée, confirmée, échouée}

Classe **`Boîtier`** *(dérivée de UC14-UC18 et du diagramme d'état)* - `idBoitier` : identifiant unique - `idAbonne` : référence au propriétaire - `latitude` : coordonnée GPS, capturée une fois à l'installation (pas de module GPS embarqué) - `longitude` : coordonnée GPS, capturée une fois à l'installation - `segmentReseau` : rattachement à la topologie électrique (préparation pour la classification sectorielle fine — transformateur/ligne — cf. section 9.2.1, piste future) - `adresseDescription` : texte libre (ex: "Poteau 12, Rue X, Quartier Y"), repère humain en complément des coordonnées - `niveauBatterie` : pourcentage ou tension batterie - `etatConnectivite` : énumération {wifi_disponible, gsm_disponible, aucun} - `etatCourant` : énumération correspondant aux états du diagramme (Initialisation, MonitoringNormal, AnomalieDetectee, ModeDegrade, DernierSouffle, HorsService) - `tableCalibration` : liste de `CalibPoint` - `derniereTransmission` : horodatage - `dateInstallation` : date

Classe **`CalibPoint`** *(déjà définie dans les échanges précédents sur le firmware)* - `adc` : valeur brute lue par l'ADC - `tensionSortie` : tension en sortie du circuit de mesure - `tensionReseau` : tension réelle du réseau correspondante

Classe **`Incident`** *(dérivée de UC19-UC22 — attributs à confirmer une fois le cycle de vie précisé)* - `idIncident` : identifiant unique - `horodatageDebut` : date/heure de détection - `horodatageFin` : date/heure de clôture (nullable) - `typeAnomalie` : type d'anomalie à l'origine - `classificationOrigine` : énumération {locale, sectorielle, globale} - `statut` : *dépend du cycle de vie de l'Incident, encore à définir* - `zoneGeographique` : zone concernée - `boitiersConcernes` : liste des boîtiers ayant contribué à la détection (relation multiple, cf. UC20) - `technicienAssigne` : référence au technicien (nullable)

Classe **`Abonné`** - `idAbonne` : identifiant unique - `nom` : nom de l'abonné - `contact` : téléphone (SMS) / email (push) - `adresse` : localisation du domicile/PME - `seuilsNotification` : seuils personnalisés (UC5)

Classe **`Technicien`** - `idTechnicien` : identifiant unique - `zoneAssignee` : zone géographique de responsabilité - `contact` : téléphone/email

Classe **`Administrateur`** - `idAdmin` : identifiant unique - `niveauAcces` : niveau de permissions

Classe **`LogAuthentification`** *(dérivée de UC13/UC19)* - `idLog` : identifiant unique - `idBoitier` : référence au boîtier concerné - `horodatage` : date/heure de la tentative - `resultat` : énumération {réussi, rejeté}

> Note : les relations entre `Abonné` / `Boîtier`, `Technicien` / `Incident`, et le statut exact de `Incident` sont maintenant proposés ci-dessous (cardinalités + cycle de vie section 10.2) — à valider avec toi.

11.2 Cardinalités retenues (proposition)

- Un **Abonné** possède 1 à plusieurs **Boîtiers**
- Un **Boîtier** génère 0 à plusieurs **Trames**
- Un **Incident** agrège 1 à plusieurs **Boîtiers** ; un **Boîtier** peut contribuer à plusieurs **Incidents**
- Une **Trame** peut être liée à 0 ou 1 **Incident**
- Un **Incident** est assigné à 0 ou 1 **Technicien** ; un **Technicien** traite plusieurs **Incidents**
- Un **Administrateur** supervise 0 à plusieurs **Boîtiers** (relation de supervision, pas de possession)
- Un **Boîtier** génère 0 à plusieurs entrées de **LogAuthentification**

11.3 Diagramme de classes

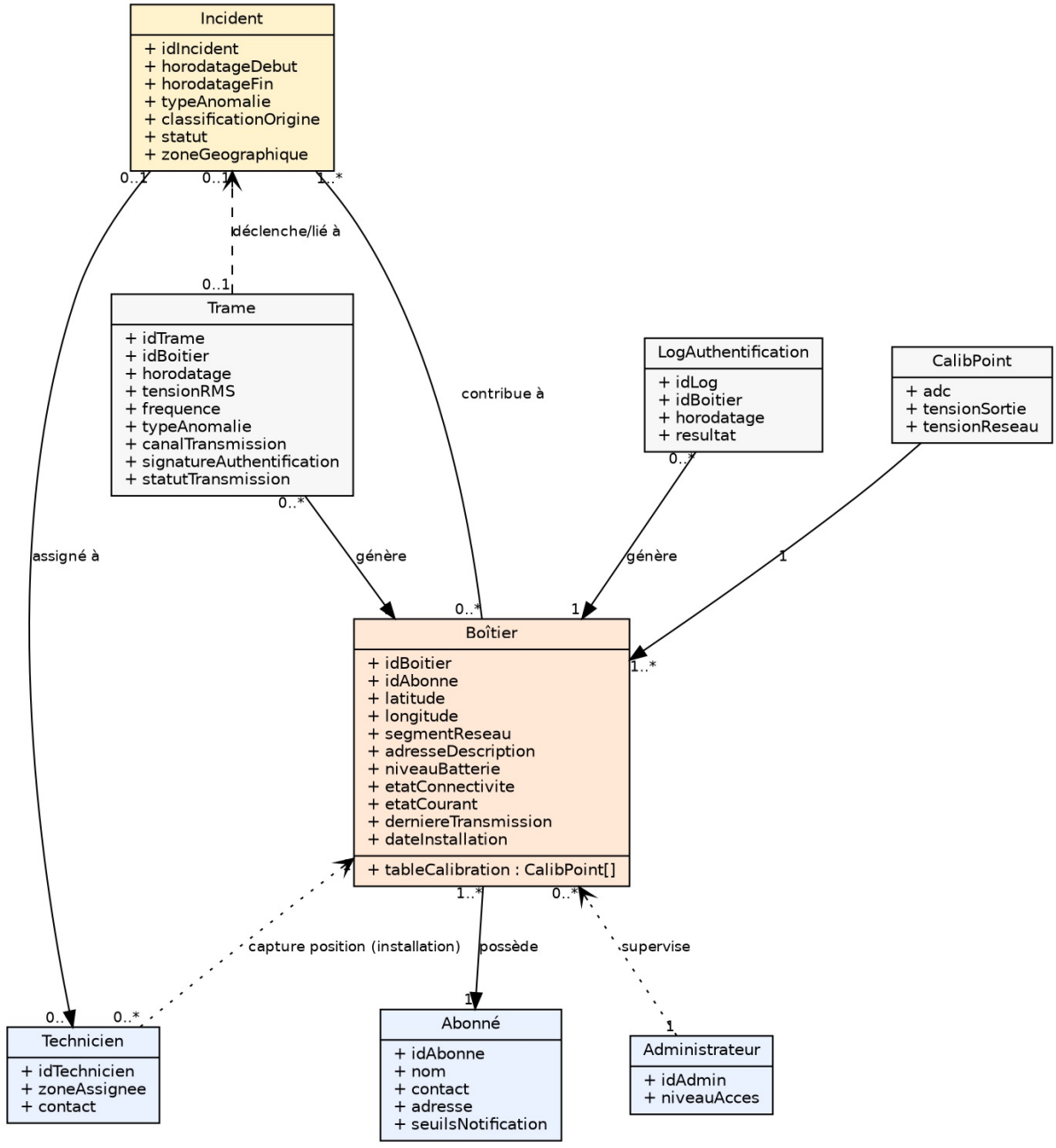


Diagramme de classes

12. Choix technologiques et justifications

12.1 Backend — Spring Boot

Choix retenu : Spring Boot (plutôt que Node.js, également maîtrisé).

Justification : - Spring Security fournit des mécanismes de sécurité robustes prêts à l'emploi (protection CSRF, JWT/OAuth2 natif, sécurité au niveau méthode), réduisant le risque d'erreur de configuration en développement solo — contrairement à Node.js/Express où ces briques (helmet, rate-limiting, validation) doivent être assemblées manuellement. - Réutilisation directe d'une expérience concrète et validée sur le projet cervarent-app (filtre JWT dans le gateway, injection `X-User-Id`, Spring Cloud Gateway, RabbitMQ), directement transposable au besoin d'authentification des trames (UC19). - Le besoin non fonctionnel critique de sécurité des données (empêcher qu'un faux boîtier n'injecte de fausses alertes) correspond exactement au terrain de prédilection de Spring Security.

12.2 Frontend Abonné — Ionic (web + mobile)

Choix retenu : Ionic, avec Angular comme framework sous-jacent (à confirmer avant démarrage).

Justification : - Un seul code source pour obtenir à la fois une version web et une version mobile, adapté au besoin de l'Abonné (UC1-UC5) qui doit être accessible sur les deux supports. - En choisissant Angular comme framework sous-jacent d'Ionic (plutôt que React/Vue), les compétences (composants, services, RxJS) sont directement réutilisables avec le Frontend Admin — une seule courbe d'apprentissage au lieu

de deux.

12.3 Frontend Admin/Technicien — Angular (web uniquement)

Choix retenu : Angular.

Justification : Adapté à une interface de supervision plus complexe et structurée (UC6-UC13), et cohérent avec le choix d'Ionic+Angular côté Abonné pour maximiser la réutilisation de compétences.

12.4 Boîtier — C++

Choix retenu : C++ (via Arduino framework / PlatformIO), plutôt que Python/MicroPython.

Justification : Les besoins non fonctionnels du boîtier (échantillonnage de plusieurs milliers de points/seconde, calcul RMS/FFT en temps réel, latence minimale) dépassent ce qu'un interpréteur Python/MicroPython peut tenir sur ESP32. C++ est le standard de facto pour ce type de traitement embarqué contraint, et cohérent avec l'environnement de développement déjà en place (Wokwi + PlatformIO).

12.5 Hébergement — GCP ou AWS

Point d'attention : Google Cloud IoT Core (service managé de connexion de devices IoT) a été définitivement fermé par Google en août 2023 — il n'est plus disponible comme option, quelle que soit la date de déploiement du projet.

- GCP reste utilisable pour héberger le backend Spring Boot (Compute Engine, Cloud Run, Pub/Sub), mais sans service IoT managé natif.
- AWS IoT Core reste actif et constitue une alternative si un bridge MQTT géré est souhaité.
- Alternative valable sur les deux clouds : auto-héberger un broker MQTT (Mosquitto, EMQX, HiveMQ) sur une VM/conteneur, offrant plus de contrôle et moins de dépendance à un service propriétaire.

Décision finale : *à trancher — en attente.*

12.6 Versionnage — GitHub

Choix retenu : GitHub. Déjà utilisé et maîtrisé, aucun changement nécessaire.

12.7 Simulation réseau — Cisco Packet Tracer

Choix retenu : Cisco Packet Tracer, en complément de Wokwi (pas en remplacement).

Justification : - Permet de simuler la topologie réseau (Wi-Fi, tours cellulaires 3G/4G, bascule entre réseaux) et de représenter visuellement le comportement de la bascule Wi-Fi/GSM (UC17) — utile pour illustrer et documenter la partie communication résiliente de l'axe de recherche RSD. - Supporte des devices IoT basiques avec simulation de serveur MQTT, permettant une représentation de haut niveau du chemin Boîtier → Backend.

Limites à garder en tête : - Ne simule pas de vrai code : ne remplace pas Wokwi, qui exécute le firmware ESP32 réel. - Ne teste pas la logique applicative réelle (authentification de trame UC19, backend Spring Boot) — outil de topologie/comportement réseau général, pas un environnement de test applicatif. - Rôle principal : visualisation, documentation, et support de soutenance — pas validation technique du système réel.